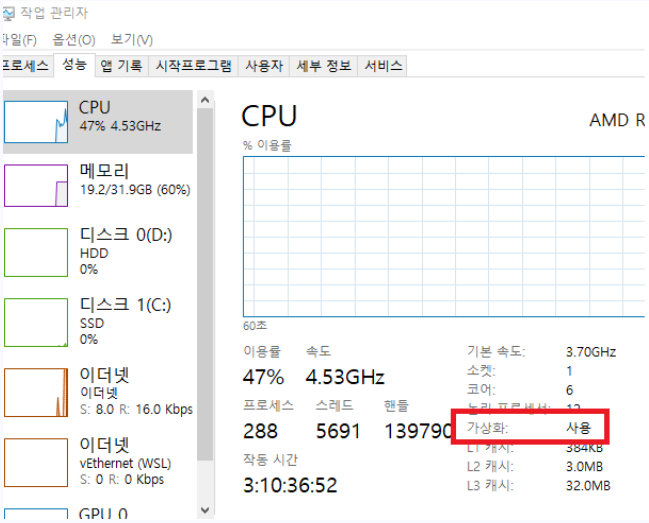
Docker Desktop 설치/접속 가이드(JupyterLab 기반)

0. 설치 전 사양 체크

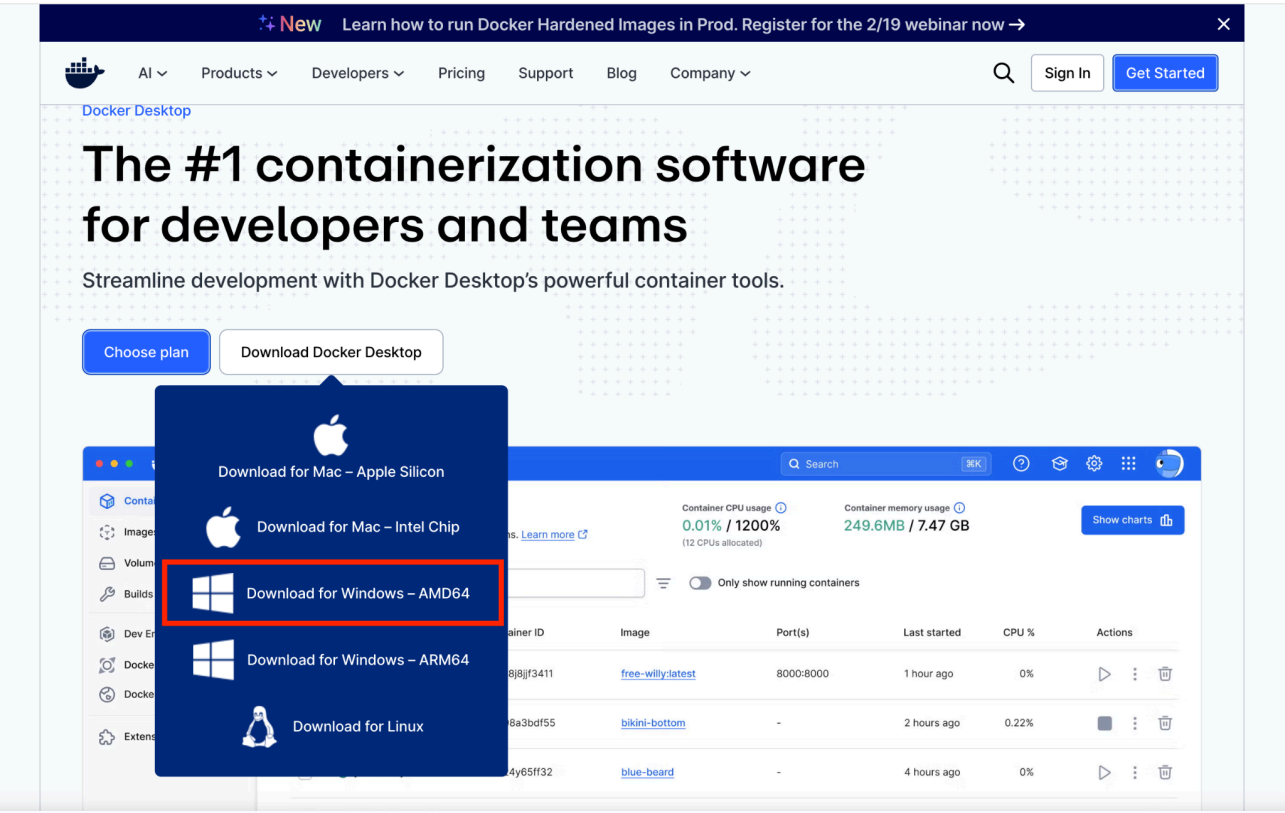
작업 관리자(Ctrl+Shift+Esc) → 성능 → CPU 항목에서 '가상화: 사용'으로 되어 있는지 확인합니다.



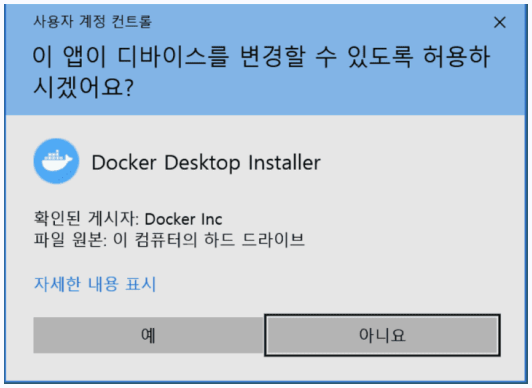
1. Docker Desktop 설치

(1) 다음 페이지로 이동해서 Download for Windows를 클릭해 Docker Desktop for Windows를 다운로드 받습니다.

링크 바로가기: [Docker Desktop 설치하러 가기]



(2) 다운로드 받은 Docker Desktop Installer.exe를 실행하면 사용자 계정 컨트롤이 나타납니다. 설치를 진행하려면 '예'를 클릭합니다.

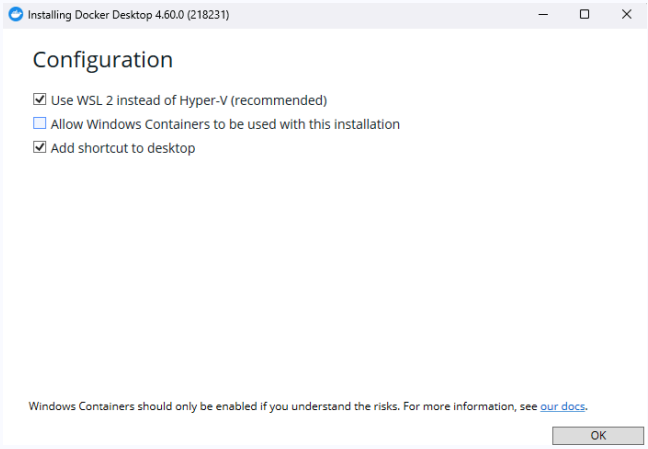


(3)안내에 따라 설치를 진행합니다. 설치 중간에 Configuration이 나타납니다. 체크하고 설치를 진행합니다.

(항목 예시1) Use WSL 2 instead of Hyper-V : Docker가 WSL2 기반으로 동작하게 설정합니다. (필수 체크)

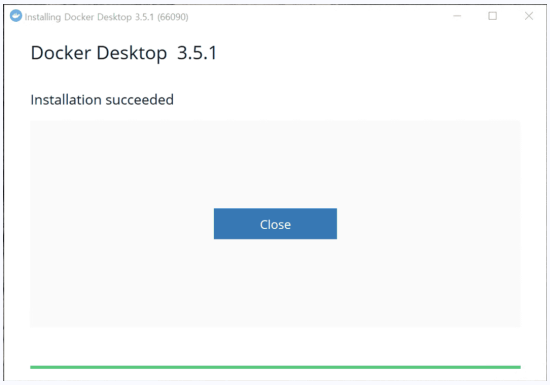
(항목 예시2) Allow Windows Containers : Windows 전용 컨테이너 사용 여부를 설정합니다.

(항목 예시3) Add shortcut to desktop : 바탕화면에 Docker 아이콘을 추가합니다. (필수 체크)



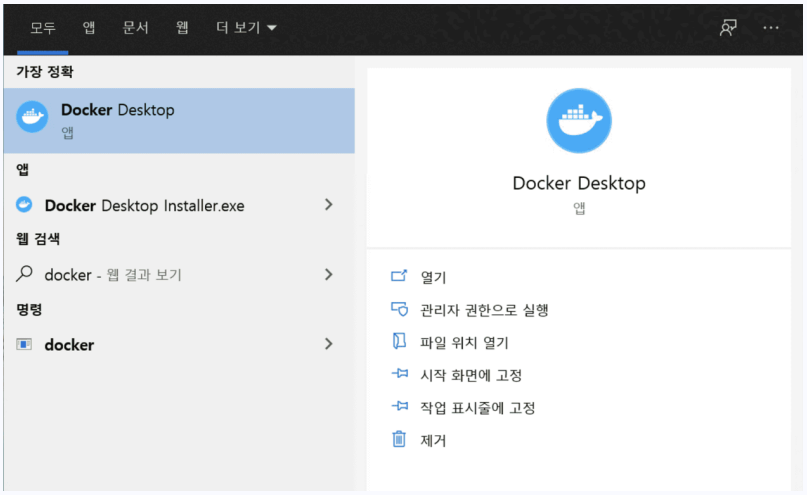
(4) Docker Desktop 설치가 진행됩니다. 몇 분 정도 시간이 걸리니 완료될 때까지 기다립니다.

(5) 설치가 끝나면 Installation succeeded 메시지가 나타납니다. 시스템 상태에 따라서 재시작이나 로그아웃을 해야하는 경우도 있습니다. Close 버튼을 클릭해 인스톨러를 종료합니다.

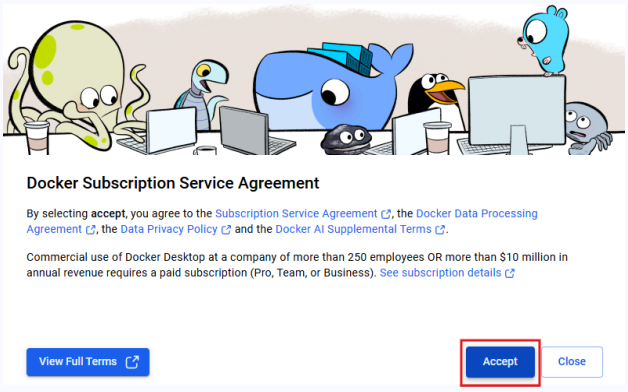


2. Docker Desktop 설정

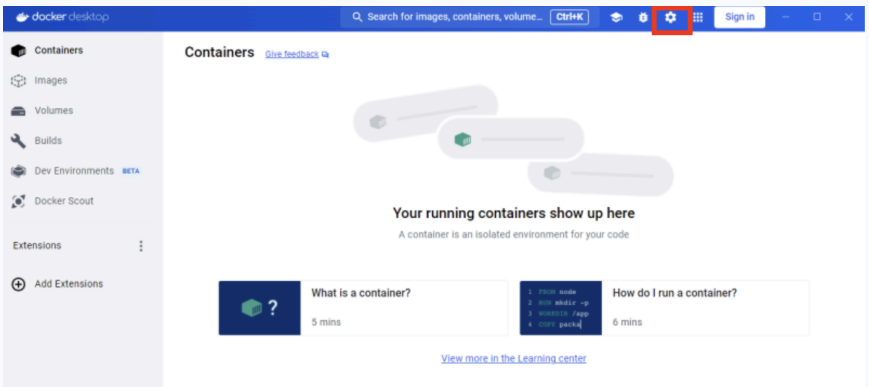
(1) 바탕화면의 Docker Desktop 아이콘이나 Windows + S로 Docker를 검색해서 Docker Desktop을 실행합니다.



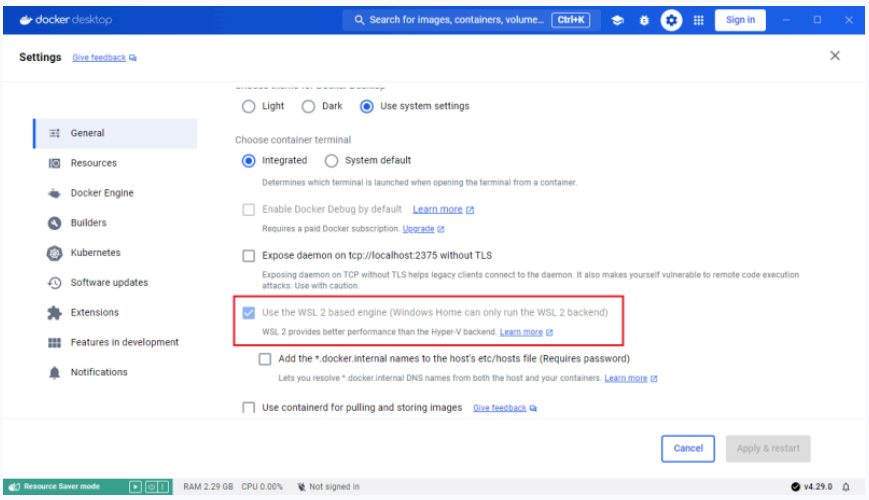
(2) Docker Desktop을 처음 실행하면 아래와 같이 약관 동의 화면과 로그인 화면이 표시됩니다. Accept 버튼을 클릭하여 약관에 동의한 후, 로그인을 진행합니다. (구글, 깃허브 연동 로그인 혹은 계정이 없는 경우 Create an account를 통해 회원가입해주세요.)



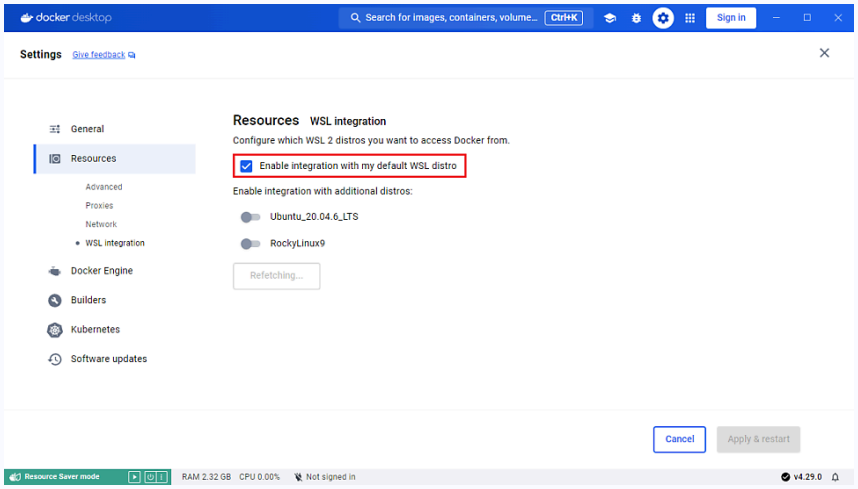
(3) 이후 셋업이 완료되고, Docker Desktop을 실행하면 아래와 같은 창이 열리며, 오른쪽 상단에 톱니바퀴 모양의 환경설정을 클릭합니다



(4) General 설정에서 Use the WSL 2 based engine에 체크가 되어 있는지 확인합니다. 이미 체크가 되어 있을 수도 있으나, 체크되어 있지 않다면 해당 항목을 선택한 후오른쪽 하단의 Apply & Restart 버튼을 클릭합니다.



(5) Resources > WSL Integration에서 Enable Integration with my default WSL distro 옵션이 활성화되었는지 확인합니다. 활성화되어있지 않았다면, 활성화한 뒤 Apply & Restart를 클릭해 Docker 엔진을 재시작합니다.



3. 실습폴더 준비 및 필수 파일 확인

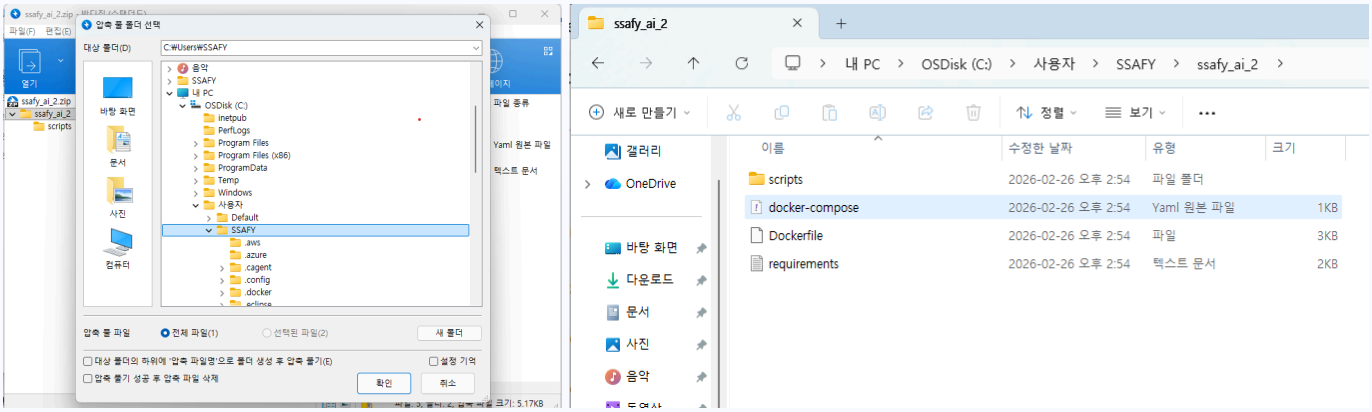
(1) PowerShell 열고 다음 명령을 입력합니다. Ubuntu-22.04가 VERSION 2로 떠야 합니다.

```
None
wsl -l -v
```

```
PS C:\Users\islove8587> wsl -l -v
NAME                   STATE           VERSION
* Ubuntu_20.04.6_LTS     Stopped         2
  docker-desktop         Running         2
  docker-desktop-data    Running         2
```

(2) 도커 환경 설정 파일 압축 해제

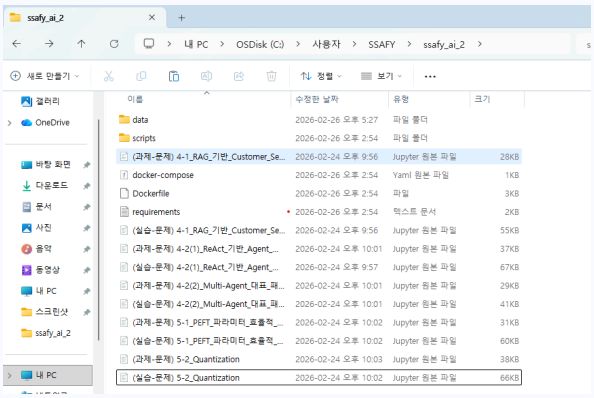
- 제공된 환경 설정 압축 파일(폴더명: ssafy_ai_2)의 압축을 해제합니다.(권장 위치: C:\Users\SSAFY)



아래의 디렉토리 구조를 확인하여 설정 파일들과 설치 스크립트(scripts/verify_env.sh)이 모두 갖춰졌는지 확인하세요.

```
None
ssafy_ai_2/

├── Dockerfile #압축파일로 제공

├── docker-compose.yaml #압축파일로 제공

├── requirements.txt #압축파일로 제공

├── scripts/  #압축파일로 제공

    └── verify_env.sh  #압축파일로 제공

├── ....# 온라인 실습실에서 실습,과제 파일들을 다운로드 받은 후, 해당 폴더에 업로드합니다.

├── (실습-문제) 4-1_RAG_기반_Customer_Service_AI_에이전트_개발

├── (과제-문제) 4-1_RAG_기반_Customer_Service_AI_에이전트_개발
```

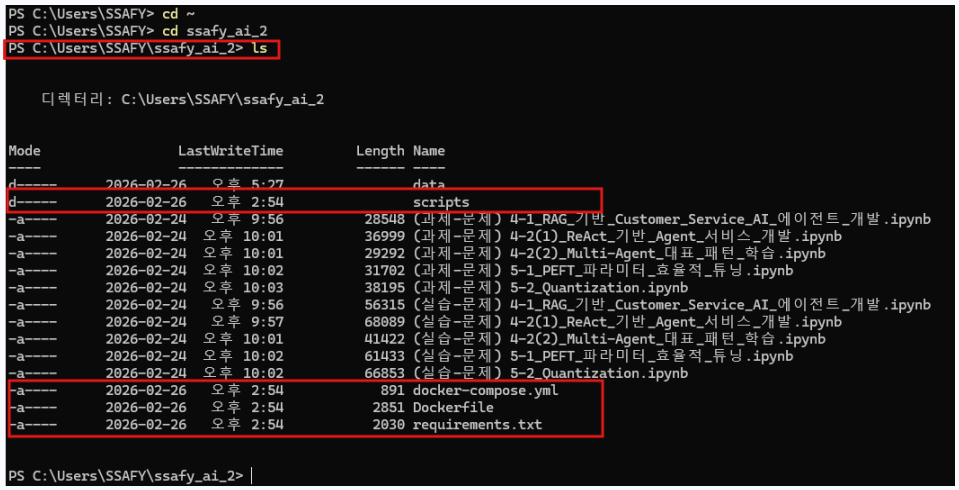
(3) 온라인 실습실에서 다운로드 받은 실습/과제 파일을 (1)에서 생성한 폴더로 이동합니다.



(4) Powershell에 접속 후, 아래 명령어를 입력하여 필수 파일 존재 여부를 확인합니다.

- 필수 파일 확인: docker-compose.yml, Dockerfile, requirements.txt

```
None
cd ~

cd ssafy_ai_2

ls
```



Q. 각 필수 파일들은 어떤 역할을 하나요?

a) docker-compose.yml: 컨테이너 실행 방식(서비스, 포트, 볼륨, 환경변수 등) 정의

b) Dockerfile: 실습용 이미지 빌드 규칙 정의

c) requirements.txt: Python 패키지 및 버전 정의

d) verify_env.sh: 환경이 정상 실행 가능한지 빠르게 진단하는 점검 스크립트

4. Docker 이미지 빌드

(1) 최초 실행 (이미지 빌드 + 컨테이너 실행)

- 최초 1회는 Dockerfile과 requirements.txt를 기준으로 이미지를 빌드하므로 시간이 다소 걸릴 수 있습니다.

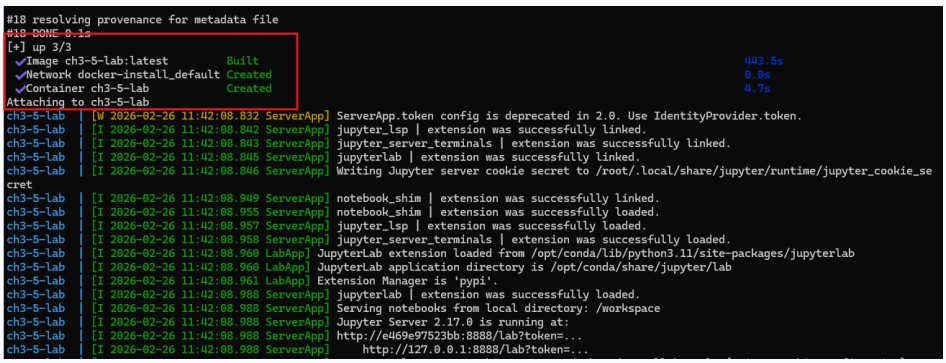
- 약 15~20분 소요되며, 네트워크 환경에 따라 예상 소요 시간은 변경될 수 있습니다.

- 반드시 Docker Desktop 프로그램이 실행된 상태에서 작업을 진행해 주시기 바랍니다.

- 처음 실행 시 패키지 설치 및 이미지 빌드가 진행됩니다.

- 빌드 완료 후 JupyterLab 실행 로그가 출력됩니다.

```
None
docker compose up --build
```



5. JupyterLab 접속

본 가이드에서는 기본 IDE로 JupyterLab 사용을 권장드립니다.

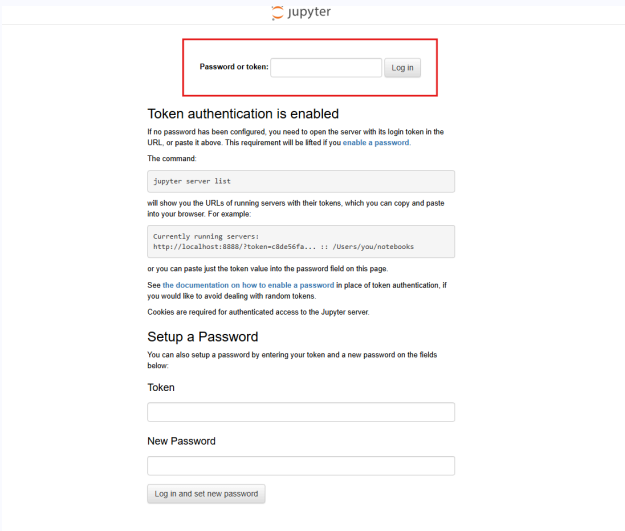
(방법1) 터미널 출력 화면 하단에서 `http://localhost:8888/tree?token=...` 형태의 URL을 확인한 후, 접속합니다. 만일 온전한 URL 형태로 보이지 않는다면, (방법2)를 시도해주세요.

(방법 2) (1) 브라우저에서 아래 주소로 접속합니다.

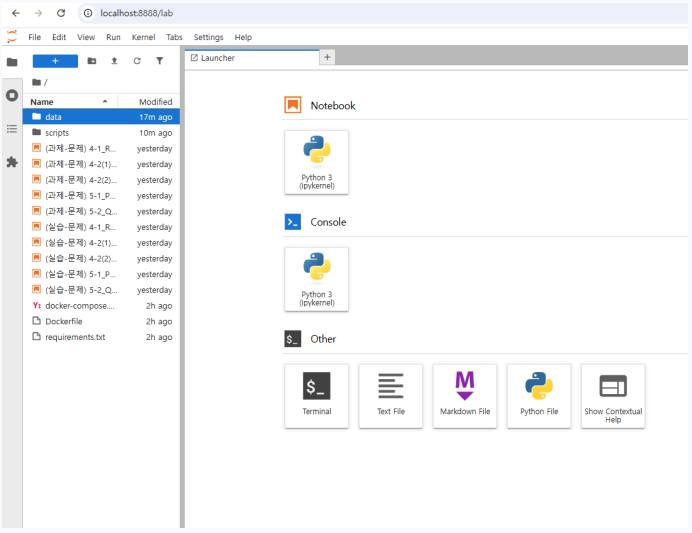
- URL: http://localhost:8888

(방법 2)(2) 토큰 입력

- JupyterLab 로그인 화면이 나오면 아래 토큰을 입력한 후,LOG IN을 클릭합니다.
- **Token**: `lab`



접속 시 화면



토큰 입력 후, 로그인 시 화면

(방법1) / (방법2) 중 가능한 방법으로 위 화면 예시와 같이 정상 접속이 되었다면, 해당 실습/과제 파일에 접속하여 학습을 시작해주세요!

6. GPU 검증

Docker 내에서g GPU 인식이 잘 되는지 검증합니다.

(0-1) 서버는 켜둔 채로 터미널 창만 되돌리고 싶을 경우, (백그라운드 전환) **GPU 검증 후, 바로 실습을 진행할 예정일 경우 해당 방식 진행**

Jupyter 웹 화면은 계속 쓰면서, 터미널에서는 다른 명령어를 치고 싶으시다면 해당 방법을 사용하세요.

- 방법: 키보드에서 알파벳 `d` 를 누르세요.

(0-2)Jupyter 서버를 완전히 끄고 검증할 경우,

실습을 마치고 컨테이너를 종료하고 싶으시다면 이 방법을 사용하세요.

- 방법: 키보드에서 `Ctrl + C` 를 누르세요.

- 설명: 한 번 누르면 "진짜 종료할 건가요?"고 묻는 메시지가 나옵니다. 이때 y를 누르세요.

(1) 서비스 상태 확인

```
None
docker compose ps
```

```
PS C:\Users\SSAFY\ssafy_ai2\docker-install> docker compose ps
NAME        IMAGE              COMMAND                  SERVICE   CREATED          STATUS          PORTS
ch3-5-lab   ch3-5-lab:latest   "bash -lc 'jupyter l…"   lab       18 minutes ago   Up 18 minutes   0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp
```

(2) 컨테이너 내부 접속

```
None
docker compose exec lab bash
```

```
PS C:\Users\SSAFY\ssafy_ai2\docker-install> docker compose ps
NAME        IMAGE              COMMAND                  SERVICE   CREATED          STATUS          PORTS
ch3-5-lab   ch3-5-lab:latest   "bash -lc 'jupyter l…"   lab       18 minutes ago   Up 18 minutes   0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp
PS C:\Users\SSAFY\ssafy_ai2\docker-install> docker compose exec lab bash
root@e469e97523bb:/workspace#
```

(3) GPU 인식 확인

- NVIDIA GPU 정보가 출력되면 정상 인식입니다.

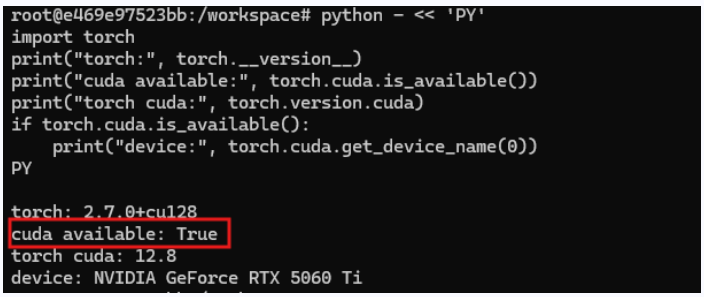
```
None
nvidia-smi
```

```
PS C:\Users\SSAFY\ssafy_ai2\docker-install> docker compose ps
NAME        IMAGE              COMMAND                  SERVICE   CREATED          STATUS          PORTS
ch3-5-lab   ch3-5-lab:latest   "bash -lc 'jupyter l…"   lab       18 minutes ago   Up 18 minutes   0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp
PS C:\Users\SSAFY\ssafy_ai2\docker-install> docker compose exec lab bash
root@e469e97523bb:/workspace# nvidia-smi
Thu Feb 26 12:02:45 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 590.57                 Driver Version: 591.86         CUDA Version: 13.1     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 5060 Ti     On  |   00000000:01:00.0  On |                  N/A |
|  0%   38C    P8              5W /  180W |     892MiB /  16311MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A             24      G   /Xwayland                                N/A   |
|    0   N/A  N/A             49      G   /Xwayland                                N/A   |
+-----------------------------------------------------------------------------------------+
```

(4) PyTorch CUDA 사용 가능 여부 확인

- 아래 명령어를 입력하여 **`cuda available: True`** 로 출력이 나타난다면, 정상 출력입니다.

```
None
python - << 'PY'

import torch

print("torch:", torch.__version__)

print("cuda available:", torch.cuda.is_available())

print("torch cuda:", torch.version.cuda)

if torch.cuda.is_available():

    print("device:", torch.cuda.get_device_name(0))

PY
```



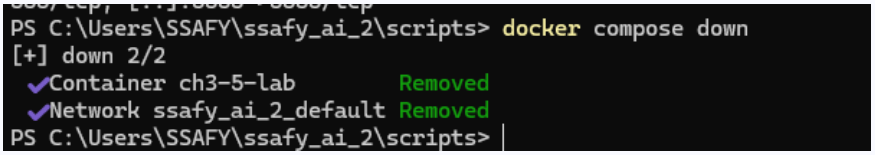
(5) 검증 후, 컨테이너 내부에서 나오기 위해 아래 명령어를 입력합니다.

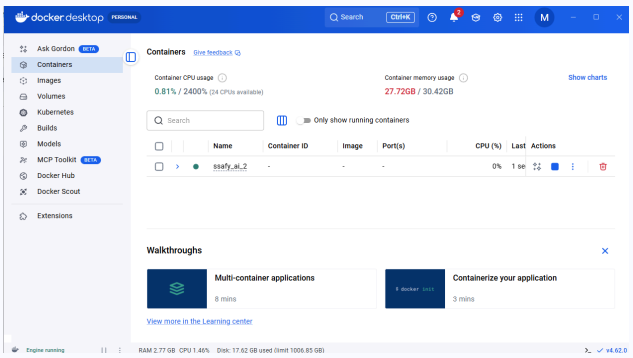
```
None
exit
```

7. 실습 종료/재시작

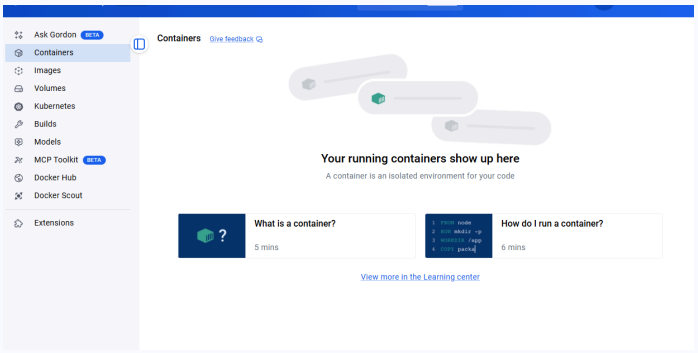
(종료 시) 아래 명령어를 입력하여 실행합니다.

```
None
docker compose down
```





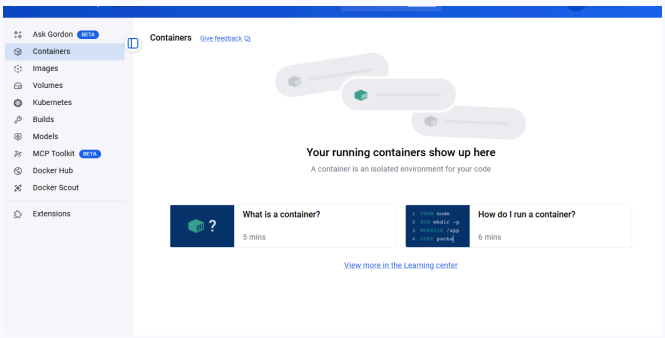
명령어 실행 전 Docker Desktop 화면



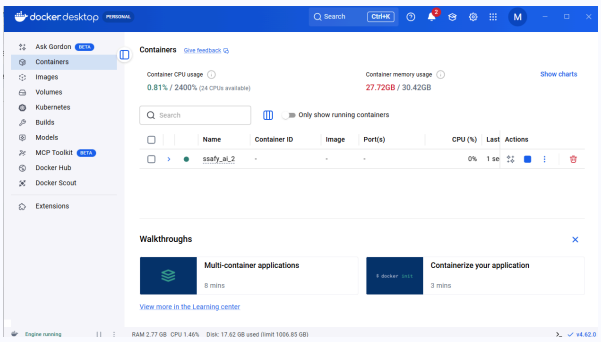
명령어 실행 후 Docker Desktop 화면

(재시작 시) 아래 명령어를 입력하여 실행합니다.

```
None
docker compose up -d
```



명령어 실행 전 Docker Desktop 화면



명령어 실행 후 Docker Desktop 화면